



Interesting Scientific Title

Your Name

Supervisors:

Faculty of Engineering

Department of Computer Science

University College London

A Project Report Presented in Partial Fulfillment of the Degree

Your degree title

September 2022

Abstract

The unreliability of reward signals is one of the core challenges for deep reinforcement learning in real-world applications. In settings such as sim-to-real fine-tuning and long-horizon robotic manipulation, reward signals are either extremely sparse, provided only upon task completion, or not fully aligned with the true objective, and therefore cannot reliably steer behavior toward the correct outcome. This thesis systematically compares, on the Door task of HumanoidBench, the fine-tuning behaviour of PPO, which uses GAE-style absolute advantage estimation, and GRPO, which uses group-relative advantage estimation, under two reward conditions: a Misaligned Reward condition that simulates unreliable reward information, and a Sparse Reward condition that simulates extremely sparse rewards.

Our experimental results show that under the Misaligned condition PPO’s return soars from a baseline of 244 to 431 while the success rate drops from 5.0% to 0.0%, exhibiting systematic reward hacking; under the Sparse condition PPO either collapses or stalls. In contrast, GRPO remains stable under both conditions, with the return held within a narrow band of 188–219. KL divergence and parameter-space distance analyses show that GRPO’s policy update is not stagnant, yet its return is decoupled from KL: the update magnitude varies with seed and condition while the return stays stable, whereas PPO’s KL and return are tightly coupled. This indicates that GRPO’s update magnitude is insensitive to the scale and density of the reward signal, and that its group-relative advantage mechanism may inherently suppress the positive-feedback loop of reward hacking, making GRPO a safer fine-tuning strategy when reward design is imperfect or rewards are extremely sparse.

Keywords— keyword 1 - keyword 2 - keyword 3

I dedicate this ...

Copyright © 2022 by Joseph Rowell
All Rights Reserved

The computer was born to solve problems that
did not exist before.

— Bill Gates

Acknowledgements

Declaration

I, Name, I declare that the thesis has been composed by myself and that the work has not be submitted for any other degree or professional qualification. I confirm that the work submitted is my own, except where work which has formed part of jointly-authored publications has been included and referenced. The report may be freely copied and distributed provided the source is explicitly acknowledged.

12/09/22

Signature

Date

Table of Contents

List of Figures	viii
List of Tables	ix
1 Project Plan	xii
2 Introduction and Background	1
2.1 Introduction	1
2.2 Related Work	3
2.2.1 Misaligned Rewards and Reward Hacking	3
2.2.2 Sparse Rewards and Long-Horizon Tasks	4
2.2.3 Pretraining–Fine-Tuning and Robot RL Post-Training	5
2.2.4 GRPO and Its Behaviour under Unreliable Rewards	5
3 Methodology	7
3.1 Problem Formulation	7
3.2 Algorithms	7
3.2.1 PPO (Proximal Policy Optimization)	7
3.2.2 GRPO (Group Relative Policy Optimization)	8
3.3 Experimental Setup	9
3.3.1 Environment	9
3.3.2 Reward Function	9
3.3.3 Baseline Model	10
3.3.4 Training and Evaluation Settings	11
3.3.5 Evaluation and Analysis Protocol	11
4 Results and Analysis	13

4.1	Main Results: Overall Performance under Two Reward Conditions	13
4.2	Diagnostic Analysis	15
4.2.1	Reward Decomposition	15
4.2.2	KL Divergence and Policy-Update Magnitude	17
4.2.3	Parameter-Space Distance	21
4.2.4	VecNormalize Drift	21
4.2.5	Summary	22
4.3	Ablation Study: The Role of the KL Penalty	23
4.4	Cross-Task Validation: The Reach Task	24
5	Discussion	26
5.1	A Possible Mechanistic Explanation of GRPO’s Robustness	26
5.2	Limitations	27
6	Conclusion	29
	References	31
	Appendix A Source Code	35
	Appendix B Project Introduction Video	36

List of Figures

4.1	Stacked-bar reward decomposition of each model on the Door task; the bar height is the accumulated value of each weighted component. Orange: door openness; blue: passing the door; grey: the remaining components. . .	16
4.2	Action-distribution KL divergence of each model relative to the Baseline, over 3 seeds; the bar height is the cross-seed mean and the dots are individual seeds.	18
4.3	Scatter plot of KL divergence versus return, two panels, left = Misaligned Reward condition, right = Sparse Reward condition; the dashed line is the baseline reference.	19
4.4	L2 distance of each model's deterministic action mean relative to the Baseline, cross-seed mean, with dots for individual seeds.	20
4.5	Average log- σ shift of each model relative to the Baseline, cross-seed mean, with dots for individual seeds.	21
4.6	Parameter-space L2 distance of each model relative to the Baseline, cross-seed mean, with dots for individual seeds.	22

List of Tables

3.1	Baseline model performance.	10
3.2	Training rewards and model naming.	11
4.1	Overall performance under two reward conditions. The PPO/GRPO entries report the cross-seed mean $\pm$ cross-seed std over the 3 training seeds (42, 98, 5075), each evaluated over 100 episodes; the Baseline is a single pretrained model, whose values are the mean over its 100 evaluation episodes.	13
4.2	Results with success-rate-based checkpoint selection. Entries are cross-seed mean $\pm$ cross-seed std over the 3 training seeds; the Baseline is a single pretrained model.	14
4.3	Weighted component contributions (cross-seed mean $\pm$ cross-seed std over 3 seeds).	15
4.4	Reward composition shares (cross-seed mean, %).	16
4.5	KL divergence of each model relative to the Baseline (cross-seed mean $\pm$ cross-seed std).	17
4.6	VecNormalize observation-statistic drift (cross-seed mean $\pm$ cross-seed std).	23
4.7	KL-penalty ablation results. The GRPO+KL column reports single-seed mean values (returns, success rates, door shares) across 3 seeds.	23
4.8	Results on the Reach task.	25

List of Algorithms

List of Abbreviations

BA: Bundle Adjustment

BRIEF: Binary Robust Independent Elementary Features

CNN: Convolutional Neural Network

DCNN: Deep Convolutional Neural Network

FAST: Features from Accelerated Segment Test

GNSS: Global Navigation Satellite System

MAP: Maximum a Posteriori

ML: Maximum Likelihood

MVS: Multi-view Stereo

ORB: Oriented FAST Rotated BRIEF

SIFT: Scale Invariant Feature Transform

SLAM: Simultaneous Localization And Mapping

SURF: Speeded-Up Robust Features

SfM: Structure from Motion

TPU: Tensor Processing Unit

TRC: TPU Research Cloud

vSLAM: visual Simultaneous Localisation and Mapping

1 | Project Plan

2 | Introduction and Background

2.1 Introduction

Deep reinforcement learning has achieved remarkable progress in robot control, yet its success depends heavily on the quality of the reward function. In practice, designing a reward function that both guides learning and is fully aligned with the true objective is extremely difficult[1]: in RL-based fine-tuning, the reward signal tends to be either too sparse, giving feedback only upon task completion[2]—especially in long-horizon manipulation tasks, where intermediate steps lack explicit reward guidance[3, 4]—or not fully aligned with the true task objective, containing misleading shaping components[5]. The common feature of these two cases is that the reward signal cannot reliably point to correct behaviour: even when the reward is dense in value, the inconsistency between a shaping reward and the true objective is itself a form of unreliable information; a high reward does not imply that the task is progressing in the right direction.

Meanwhile, pretraining–fine-tuning has become an important paradigm for building robotic policies[6], in which a general policy is first pretrained on large-scale data and then fine-tuned on a target task to adapt it to a specific embodiment and task; RL-based fine-tuning is a key component of this process. PPO[7] is currently the most widely used on-policy RL algorithm and is often the default choice for fine-tuning. Its advantage estimation relies on GAE[8]: if a shaping component consistently increases the return, then even if it conflicts with the final task objective, it can exert persistent optimization pressure. Consequently, when the reward condition during fine-tuning deviates from the true objective, PPO’s optimization direction may be pulled by misleading reward components.

In recent years, GRPO has been widely adopted and achieved significant success as an alternative in RL fine-tuning of LLMs[9]. Unlike PPO, which takes raw reward maximization as its optimization basis, GRPO changes the way rewards are compared, shifting from

absolute improvement to within-group relative improvement: when a reward component is generally increased across a group of samples, within-group normalization weakens its role as a discriminating signal, so GRPO may become less sensitive to systematic reward inflation[10]. In robot fine-tuning, an increased reward does not necessarily imply task progress, and this difference may make GRPO more insensitive to systematic reward growth that is inconsistent with the task objective. However, the application of GRPO in robotics is still in its infancy, and a systematic comparison with PPO under unreliable reward conditions in fine-tuning is still lacking.

To fill this gap, this thesis systematically compares the fine-tuning behaviour of PPO and GRPO on the Door task of HumanoidBench[11] under two reward conditions: a Misaligned Reward condition that simulates unreliable reward information, and a Sparse Reward condition that simulates extremely sparse rewards. We find that PPO undergoes systematic reward hacking when rewards are unreliable, and collapses or stalls when rewards are sparse; GRPO remains stable under both conditions, with its policy-update magnitude varying with seed and condition while its return stays stable. Diagnostic analysis shows that this stability stems from the normalization effect of group-relative advantage; KL-penalty ablation and cross-task validation further confirm the source and generality of this mechanism. Our main contributions are:

1. We reveal the empirical reward-hacking behaviour of PPO under unreliable rewards. Under the Misaligned condition PPO’s return soars from a baseline of 244 to 431 while the success rate drops from 5.0% to 0.0%; under the Sparse condition it either collapses or stalls, with the failure mode switching with the reward condition.
2. We demonstrate the stability of GRPO under both reward conditions and its KL-return decoupling. GRPO’s return is stable within a narrow band of 188–219 across all seeds; its update magnitude varies with seed and reward condition, yet its return is decoupled from KL and does not drift with the update magnitude.

3. Through reward decomposition and KL-penalty ablation we diagnose the source of the stability mechanism. Component-wise reward decomposition shows that PPO’s return gain is concentrated in components weakly related to success; the KL-penalty ablation rules out the alternative explanation that “stability stems from the absence of the KL penalty,” attributing stability to the normalization effect of group-relative advantage.
4. We provide cross-task evidence and practical implications. Additional experiments on the Reach task show that the conclusions are not limited to the Door task; in scenarios where reward design is imperfect or rewards are extremely sparse, GRPO is a more robust fine-tuning choice than PPO, and model selection should jointly monitor reward component decomposition and success rate rather than return increases alone.

2.2 Related Work

2.2.1 Misaligned Rewards and Reward Hacking

The goal of reinforcement learning is set by the reward function. However, in real tasks the reward function often cannot perfectly describe the true objective and can only serve as an imperfect proxy for it; over-optimizing such an imperfect proxy ends up harming performance on the true objective[5]. When the reward design is not fully aligned with the true objective, an agent may systematically exploit loopholes in the reward function and optimize a surrogate metric instead of the true objective; this phenomenon is called reward hacking[12]. Skalse et al.[13] provide a formal definition: optimizing an imperfect proxy causes a drop in performance under the true reward. Everitt et al.[14] were the first to formalize this problem as a Corrupt Reward MDP, pointing out that reward signals may be corrupted by, e.g., sensor errors, and that traditional RL methods struggle to

compensate. Pan et al.[15] show in a systematic study across four misspecified-reward environments that stronger agents are more prone to exploit misspecified rewards, and that a phase transition exists in which true rewards drop sharply once capability exceeds a threshold. As model scale grows, reward hacking is equally prevalent in LLMs and multimodal models; the survey by Wang et al.[16] unifies these phenomena under the view of high-dimensional objectives being compressed into reward representations. In continuous control and robot manipulation tasks, every component of reward shaping is a potential target of exploitation, making the evaluation and mitigation of reward hacking an important problem for safe reinforcement learning.

2.2.2 Sparse Rewards and Long-Horizon Tasks

When the reward signal is provided only upon final task completion, the agent receives no learning signal on the vast majority of time steps; this setting is called a sparse reward[2]. It introduces two kinds of difficulty: exploration efficiency is extremely low, since random exploration can rarely reach a successful state[3]; and credit assignment is hard, since even on accidental success it is difficult to tell which early decisions led to the success[4]. When the task further has a long-horizon, multi-stage structure, both difficulties are amplified[3, 4]. The robotics community has therefore developed a variety of methods to recover learning signals, such as hindsight experience replay[2], guidance from demonstrations[17], reuse of off-the-shelf base controllers[3], action chunking with intrinsic rewards[4], curriculum learning[18], curiosity-driven exploration[19, 20], and hierarchical structure[21]. In contrast, our Sparse Reward setting deliberately avoids such additional guidance and directly investigates whether the choice of the policy-optimization algorithm itself affects the robustness of a policy to extremely sparse feedback.

2.2.3 Pretraining–Fine-Tuning and Robot RL Post-Training

In this paradigm, RL post-training is a key tool in the fine-tuning stage. On the one hand, RL has been used to train vision–language–action foundation models: SimpleVLA-RL[22] scales the training of a VLA model via online reinforcement learning, and ReinFlow[23] fine-tunes a flow-matching policy with online RL. On the other hand, purely RL-pretrained policies also rely on fine-tuning to adapt to new environments, e.g., large-scale SAC pretraining followed by model-based fine-tuning[24] and transformer sequence-modeling pretraining followed by RL fine-tuning over uneven terrain[25]. Robot Trains Robot[26] further studies fine-tuning in the real world, where an arm teacher assists a humanoid student. The survey by Gu et al.[27] systematically reviews these learning-based methods in humanoid locomotion and manipulation. These works are usually evaluated on their own collected data and benchmarks, and a unified humanoid environment for systematically studying behaviour in the fine-tuning stage is lacking. HumanoidBench[11] provides a standardized evaluation environment for this paradigm; the Door task used in this thesis comes from this benchmark. In these paradigms, fine-tuning a pretrained policy on a target environment is common practice, but the reward condition during fine-tuning may deviate from that during pretraining, and the effect of such a deviation on policy stability has not been systematically studied.

2.2.4 GRPO and Its Behaviour under Unreliable Rewards

GRPO was originally proposed for mathematical reasoning in LLMs[9]; its group-based comparison mechanism is naturally robust to individual-level reward noise. Noise-corrected GRPO[28] analyzes this property theoretically and proposes a noise-correction method to obtain unbiased gradients; Yao et al.[10] observe in multimodal RL alignment experiments that GRPO is the most reward-hacking-resistant among three compared algorithms. In continuous control, Khanda et al.[29] present the first theoretical framework

extending GRPO to continuous control, with components including trajectory-clustering grouping and state-aware advantage estimation. De Oliveira et al.[30] observe in classical RL environments that, on long-horizon continuous-control tasks, critic-free GRPO is usually inferior to PPO, with exceptions only in short-horizon environments, indicating that GRPO’s advantage is not universal. Doorman[31] applies GRPO to sim-to-real fine-tuning of a humanoid Door task to mitigate partial observability under binary success signals. Taken together, existing work mainly focuses on whether GRPO can be used for continuous control or specific robot scenarios, or on comparing GRPO and PPO when used directly for training from scratch; although some work notices the application of GRPO in fine-tuning scenarios, a systematic study of the relative behaviour of GRPO and PPO under unreliable and extremely sparse rewards is still lacking. This thesis addresses this gap by systematically comparing the relative behaviour of the two algorithms in a fine-tuning setting, and further analyzing the relationship between group-relative advantage and the stability of policy updates.

3.1 Problem Formulation

We consider the standard reinforcement learning framework, in which an agent executes actions $a_t \sim \pi_\theta(\cdot \mid s_t)$ in an environment, receives rewards r_t , and maximizes the cumulative discounted return $\mathbb{E}[\sum_t \gamma^t r_t]$. In the policy fine-tuning scenario, we start from a pretrained policy π_0 and continue optimization in a target environment whose reward function $\hat{R}$ may differ from the reward function R used during training.

We focus on two reward conditions of practical interest:

- **Misaligned Reward:** $\hat{R}$ is a weighted sum of multiple components, some of which are not fully aligned with the true success condition. The reward is non-zero at every step, but its magnitude does not necessarily reflect task progress.
- **Sparse Reward:** during training, $\hat{R}$ provides a binary signal only at trajectory termination, equal to 1 for success and 0 otherwise. The reward is fully aligned with the objective, but carries very little information. The two conditions differ only in the training reward; at evaluation time both are evaluated with the original full weighted reward so that returns are comparable across conditions.

3.2 Algorithms

3.2.1 PPO (Proximal Policy Optimization)

PPO estimates advantages using GAE:

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (3.1)$$

where δ_t is the TD error and λ controls the bias–variance trade-off. The GAE advantage directly depends on the absolute magnitude of the reward: any behaviour that makes $\delta_t > 0$ produces a positive learning signal.

3.2.2 GRPO (Group Relative Policy Optimization)

Following the trajectory-level design of Doorman[31], we use group-relative advantages for humanoid policy fine-tuning. Limited by single-GPU memory, we use $K = 4$ parallel environments, each independently sampling one rollout to form a group, obtaining discounted returns $\{G_1, \dots, G_K\}$ with group mean $\bar{G} = \frac{1}{K} \sum_k G_k$. The GRPO advantage is:

$$\hat{A}_k^{\text{GRPO}} = \frac{G_k - \bar{G}}{\sigma_G + \epsilon} \quad (3.2)$$

where σ_G is the standard deviation of within-group returns. This advantage is then fed into PPO’s standard clipped objective:

$$\mathcal{L}^{\text{GRPO}} = \mathbb{E} \left[\min \left(\rho_k \hat{A}_k^{\text{GRPO}}, \text{clip}(\rho_k, 1 - \epsilon, 1 + \epsilon) \hat{A}_k^{\text{GRPO}} \right) \right] \quad (3.3)$$

where $\rho_k = \frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}$ is the importance-sampling ratio. Unlike the original LLM version of GRPO, our implementation does not use the KL penalty term $\beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$; the trust region is provided solely by PPO’s clipping mechanism. In addition, GRPO does not use a value function (critic); the advantage signal is produced entirely by within-group relative comparison, so no absolute value estimate needs to be learned, which also avoids the extra signal noise from value-function estimation error.

The key difference between the two algorithms lies in the source of the advantage signal: PPO’s GAE is based on absolute reward values, so any reward increase is encoded as a positive advantage; GRPO’s group-relative advantage is based on within-group relative performance, so only behaviour that outperforms other samples in the same group yields a positive advantage.

3.3 Experimental Setup

3.3.1 Environment

- Task: HumanoidBench ‘h1hand-door-v0’, the H1 humanoid pushing a door open and passing through it.
- Observation: 155-dimensional vector, normalized by VecNormalize before being fed to the policy.
- Action space: 61-dimensional continuous control (diagonal Gaussian policy).
- Maximum steps: 1000 steps/episode.
- Success criterion: the robot’s torso x-coordinate > 0.8 , i.e., crossing the door frame located at $x = 0.8$ m.

The environment uses the MuJoCo physics engine at a control frequency of 50 Hz (frame_skip = 10). The task is difficult because the robot must coordinate whole-body motion, first approach the door and place its hand on the handle to press it, then push the door open and pass through.

3.3.2 Reward Function

The original Door reward is composed of 6 raw signals forming 5 weighted terms:

$$R = \underbrace{0.1 \cdot R_{\text{stand}} \cdot R_{\text{control}}}_{\text{posture}} + \underbrace{0.45 \cdot R_{\text{door_open}}}_{\text{door openness}} + \underbrace{0.05 \cdot R_{\text{hatch_open}}}_{\text{handle press}} + \underbrace{0.05 \cdot R_{\text{hand_prox}}}_{\text{hand-handle distance}} + \underbrace{0.35 \cdot R_{\text{passage}}}_{\text{passing the door}} \quad (3.4)$$

Each component is computed as follows:

- R_{stand} : equals 1 when the torso height ≥ 1.65 m and the uprightness ≥ 0.9 , decaying when deviating; range $[0, 1]$.

- $R_{\text{control}} = (4 + \bar{c})/5$, where $\bar{c}$ is the reward value of the joint-torque interval; smaller torque brings it closer to 1, so $R_{\text{control}} \in [0.8, 1]$.
- $R_{\text{door_open}} = \min(1, q_d \cdot |q_d|)$, where q_d is the door joint angle (rad); for a positive opening it degenerates to $\min(1, q_d^2)$, saturating at 1 when $q_d = 1$ rad, and the door can be pushed or pulled every step to produce a non-zero signal.
- $R_{\text{hatch_open}}$: equals 1 when the handle joint angle lies within $[0.75, 2]$ rad.
- $R_{\text{hand_prox}}$: equals 1 when the minimum distance of the two hands to the handle is ≤ 0.25 m.
- R_{passage} : equals 1 when the torso (imu site) x-coordinate ≥ 1.2 m, rising linearly as x goes from 0.2 to 1.2 m, and 0 when $x \leq 0.2$ m.

3.3.3 Baseline Model

The baseline model is obtained by pretraining PPO on the Door task with curriculum learning. The curriculum consists of four stages that gradually increase task difficulty, using the full dense reward during training. The baseline performance is:

Metric	Value
Success rate	5.0%
Mean return	244.0

Table 3.1: Baseline model performance.

The baseline is a single pretrained model; the values are the mean over its 100 evaluation episodes, with no cross-seed statistics. Although the baseline success rate is low, it already has basic structural understanding of the task, which makes it a suitable common starting point for comparing the stability of the two fine-tuning strategies.

3.3.4 Training and Evaluation Settings

The two reward conditions simulate different challenges of reward design in real RL fine-tuning; their definitions are given in Section ??, and the training rewards and corresponding model names are:

Condition	Training reward	PPO model	GRPO model
Misaligned Reward	Original dense reward (5 terms)	PPO (Misaligned)	GRPO (Misaligned)
Sparse Reward	Binary reward (Success=1, else 0)	PPO (Sparse)	GRPO (Sparse)

Table 3.2: Training rewards and model naming.

All models are initialized from the baseline weights with a learning rate of 3×10^{-5} , 4 parallel environments, and 5M training steps. The model selection criterion is to choose, among the periodic evaluations during training, the checkpoint with the highest mean reward under the Misaligned condition and the highest mean reward under the Sparse condition, respectively. All experiments are repeated over 3 independent training seeds (42, 98, 5075) to verify statistical robustness.

Evaluation uses 5 independent seeds (1024, 2024, 777, 88, 13), each running 20 episodes, for a total of 100 episodes per model. All inference uses a deterministic policy (‘deterministic=True’). The main evaluation metrics are the success rate and the episode return.

3.3.5 Evaluation and Analysis Protocol

Evaluation metrics. The success rate is defined as the proportion of the 100 evaluation episodes in which the robot successfully passes through the door frame; the success criterion is given in Section ?. The episode return is the cumulative weighted reward of a single episode.

Reward decomposition. We capture each reward component in the ‘info’ dict of each step in the environment wrapper and output the accumulated value of each component

at episode termination. For the posture-maintenance component ($0.1 \cdot R_{\text{stand}} \cdot R_{\text{control}}$), since $\sum(a_t \cdot b_t) \neq (\sum a_t) \cdot (\sum b_t)$, we accumulate the products step by step rather than summing over the episode and then multiplying.

KL divergence and policy analysis. We collect 1000 raw observations by rolling out the baseline policy in the environment. For each fine-tuned model, we normalize the observations with its own VecNormalize statistics and then extract the action-distribution parameters $(\mu, \log \sigma)$. For two d -dimensional diagonal Gaussian distributions P and Q , the closed-form KL divergence is:

$$D_{\text{KL}}(P \parallel Q) = \sum_{i=1}^d \left[\log \frac{\sigma_{Q,i}}{\sigma_{P,i}} + \frac{\sigma_{P,i}^2 + (\mu_{P,i} - \mu_{Q,i})^2}{2\sigma_{Q,i}^2} - \frac{1}{2} \right] \quad (3.5)$$

We compute $D_{\text{KL}}(\pi_{\text{finetuned}} \parallel \pi_{\text{baseline}})$ at each observation and report its mean, median, standard deviation and quantiles. The parameter-space distance is computed directly on the model weights as the per-layer L2 norm $\|\theta_{\text{finetuned}} - \theta_{\text{baseline}}\|_2$.

4 | Results and Analysis

4.1 Main Results: Overall Performance under Two Reward Conditions

Metric	Baseline	PPO (Misaligned)	GRPO (Misaligned)	PPO (Sparse)	GRPO (Sparse)
Success rate	5.0%	0.0% $\pm$ 0.0%	1.3% $\pm$ 1.2%	0.3% $\pm$ 0.6%	1.3% $\pm$ 1.5%
Mean return	244.0	431.4 $\pm$ 8.0	210.5 $\pm$ 9.1	183.6 $\pm$ 56.5	193.6 $\pm$ 6.5

Table 4.1: Overall performance under two reward conditions. The PPO/GRPO entries report the cross-seed mean $\pm$ cross-seed std over the 3 training seeds (42, 98, 5075), each evaluated over 100 episodes; the Baseline is a single pretrained model, whose values are the mean over its 100 evaluation episodes.

Misaligned Reward condition. PPO’s return soars to 431 on all 3 seeds, a 77% increase over the baseline, yet the success rate consistently drops to 0%, fully decoupling the return increase from task completion. This is the defining signature of reward hacking: PPO learns to maximize return rather than to complete the task. GRPO’s return is stable at 211, only 13% below the baseline, and it does not over-respond to this unreliable reward signal.

Sparse Reward condition. After removing the dense reward, PPO’s behaviour diverges sharply across training seeds: seed 42’s return collapses to 118, 52% below the baseline, while seeds 98 and 5075 barely maintain 214–219, giving a cross-seed std as high as 56.5. Its behaviour is dominated by training stochasticity rather than task structure, indicating that PPO cannot obtain a stable learning signal under sparse rewards. GRPO’s return is stable at 188–201 across all seeds, with a cross-seed mean of 194, without the collapse or seed divergence seen for PPO.

The cross-condition comparison shows that PPO’s return drops from 431 (Misaligned) to 184 (Sparse), while GRPO drops only slightly from 211 to 194. The key implication is

that GRPO is not locked into reward-hacking behaviour under the Misaligned condition, so it does not need to recover when the reward structure switches; whereas PPO’s failure mode switches with the condition: hacking when the reward is misleading, and collapsing or stalling when the reward is sparse. The three seeds and the two conditions corroborate each other, indicating that these differences are systematic algorithmic-mechanism behaviours rather than accidental artifacts of individual seeds.

From a worst-case-seed perspective, the difference is equally pronounced: PPO’s lowest return under the Sparse condition is 118.4, 52% below the baseline, while GRPO’s lowest is 188.1, only 23% below; under the Misaligned condition every GRPO seed has a return of at least 200.9, whereas all three PPO seeds achieve a 0% success rate despite high returns. In terms of the worst-case lower bound of fine-tuned performance, GRPO is significantly better than PPO, which is the direct basis for its being a safer fine-tuning option.

To rule out the influence of the checkpoint-selection metric on our conclusions, we additionally run a set of experiments using the success rate rather than the mean reward as the model-selection criterion, with all other settings identical to Section 3.3. The main results are:

Metric	Baseline	PPO (Misaligned)	GRPO (Misaligned)	PPO (Sparse)	GRPO (Sparse)
Success rate	5.0%	0.7% $\pm$ 0.6%	1.7% $\pm$ 2.1%	0.3% $\pm$ 0.6%	1.3% $\pm$ 1.5%
Mean return	244.0	431.2 $\pm$ 38.8	191.4 $\pm$ 28.9	183.6 $\pm$ 56.5	193.6 $\pm$ 6.5

Table 4.2: Results with success-rate-based checkpoint selection. Entries are cross-seed mean $\pm$ cross-seed std over the 3 training seeds; the Baseline is a single pretrained model.

Under the Sparse Reward condition, the checkpoints selected by success rate and by mean reward give identical results, a direct consequence of the two metrics being equivalent under a binary reward. Under the Misaligned Reward condition, even when the success rate is used as the criterion, PPO’s return stays as high as 431, with seed 42’s checkpoint reaching 476 yet a success rate of only 0.7%, consistent with the low-success pattern under

mean-reward selection. This shows that reward hacking is a systematic result of PPO’s optimization process rather than an artifact of the checkpoint-selection metric. Under by-success selection GRPO’s return is 191 and its success rate 1.7%, remaining near the baseline and showing no reward hacking. Overall, the main conclusions are robust to the checkpoint-selection metric: PPO’s reward hacking and Sparse-condition instability, and GRPO’s conservative stability, do not depend on the model-selection method.

4.2 Diagnostic Analysis

The main results show that PPO engages in reward hacking when rewards are unreliable, while GRPO remains stable. In this section we further localize the source of the difference through three dimensions of diagnostic analysis: reward decomposition, KL divergence, and parameter-space distance.

4.2.1 Reward Decomposition

Comparison of the weighted contributions of each model (cross-seed mean $\pm$ cross-seed std over 3 seeds):

Component (weight)	Baseline	PPO (Misaligned)	GRPO (Misaligned)	PPO (Sparse)	GRPO (Sparse)
Posture (0.1)	61.6 $\pm$ 0.0	84.4 $\pm$ 1.3	61.3 $\pm$ 2.5	56.5 $\pm$ 3.8	61.3 $\pm$ 2.5
Door openness (0.45)	131.1 $\pm$ 0.0	263.4 $\pm$ 8.1	97.4 $\pm$ 8.4	81.2 $\pm$ 47.3	97.4 $\pm$ 8.4
Handle press (0.05)	17.7 $\pm$ 0.0	39.2 $\pm$ 0.4	18.2 $\pm$ 0.6	15.4 $\pm$ 4.0	18.2 $\pm$ 0.6
Hand–handle dist. (0.05)	32.0 $\pm$ 0.0	44.1 $\pm$ 0.5	31.8 $\pm$ 1.1	28.2 $\pm$ 4.2	31.8 $\pm$ 1.1
Passing the door (0.35)	1.7 $\pm$ 0.0	0.2 $\pm$ 0.1	1.8 $\pm$ 1.4	2.3 $\pm$ 2.5	1.8 $\pm$ 1.4

Table 4.3: Weighted component contributions (cross-seed mean $\pm$ cross-seed std over 3 seeds).

Change in the reward composition shares (cross-seed mean):

Under the Misaligned condition, PPO’s total reward increment over the baseline is +187 (cross-seed mean), of which the door-openness increment is +294 (unweighted), while

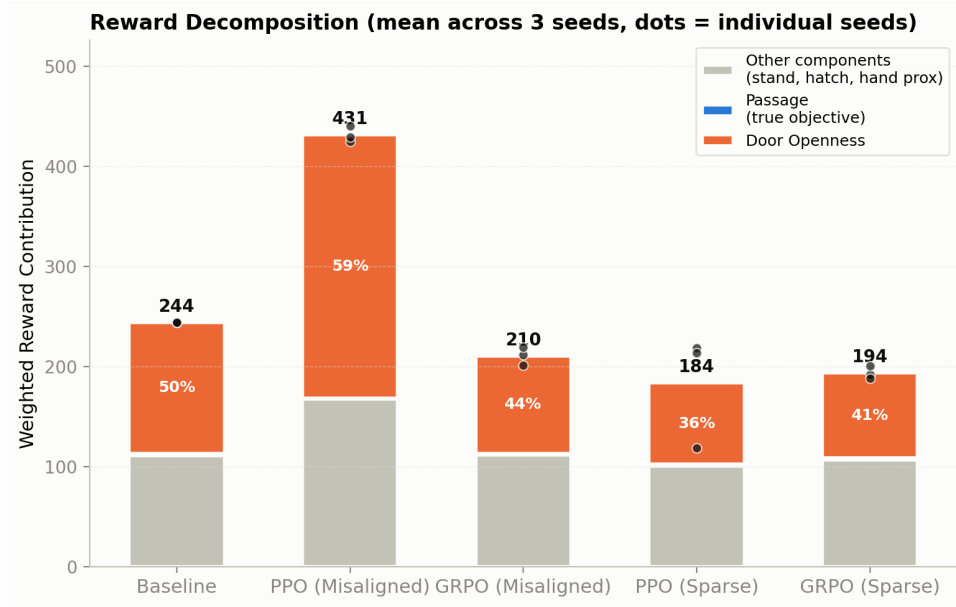


Figure 4.1: Stacked-bar reward decomposition of each model on the Door task; the bar height is the accumulated value of each weighted component. Orange: door openness; blue: passing the door; grey: the remaining components.

Component	Baseline	PPO (Misaligned)	GRPO (Misaligned)	PPO (Sparse)	GRPO (Sparse)
Posture	27.1%	21.1%	30.3%	37.0%	31.8%
Door openness	49.8%	58.5%	43.7%	35.9%	41.1%
Handle press	8.4%	9.4%	9.4%	8.8%	10.0%
Hand–handle dist.	14.2%	10.9%	15.7%	16.7%	16.4%
Passing the door	0.4%	0.1%	0.9%	1.6%	0.7%

Table 4.4: Reward composition shares (cross-seed mean, %).

the passing-the-door reward changes by -4.1 ; the door-openness share correspondingly rises from 49.8% to 58.5%, and the passing-the-door share falls from 0.4% to 0.1%. This shows that most of PPO’s return gain comes from manipulating the door angle, while the reward that truly reflects task completion (passing the door) actually decreases, decoupling return growth from the task goal — a typical reward-hacking signature. Notably, the amplification is not limited to door openness: posture rises from 61.6 to 84.4 and handle press from 17.7 to 39.2, both increasing significantly, with only the passing-the-door component decreasing. This indicates that PPO’s hacking tendency is generic: any

component that is available at every step and weakly related to the success condition is amplified. Door openness is amplified most strongly because its weight (0.45) is the highest, and because the door can be opened and closed repeatedly, giving a non-zero every-step reward with dense gradients, providing the largest return increment among equally accessible components; in contrast, the passing-the-door reward is near zero on the vast majority of trajectories and cannot provide effective gradients. This behaviour appears consistently across all seeds, with the cross-seed std of PPO’s door_openness being only 8.1. In contrast, GRPO (Misaligned)’s door-openness share (43.7%) is even lower than the baseline, showing that it not only fails to amplify this component but relatively suppresses it; under the Sparse condition both models have shares below the baseline (PPO 35.9%, GRPO 41.1%), with no over-reliance on a single reward component. This also shows that whether reward hacking occurs depends on the algorithmic mechanism rather than on the reward weight structure itself.

4.2.2 KL Divergence and Policy-Update Magnitude

On the same fixed 1000 observations collected by the baseline policy, we compute the KL divergence of each model’s action distribution relative to the Baseline using the closed-form diagonal-Gaussian solution. The following table reports the cross-seed mean $\pm$ std over 3 seeds:

Model	Mean KL (nats)	Per-dim (nats/dim)	Per-seed
PPO (Misaligned)	32.6 ± 6.9	0.53	25.4, 33.2, 39.2
GRPO (Misaligned)	11.6 ± 7.8	0.19	19.4, 11.7, 3.8
PPO (Sparse)	18.0 ± 25.5	0.29	47.4, 3.7, 2.8
GRPO (Sparse)	16.4 ± 10.9	0.27	19.6, 4.2, 25.2

Table 4.5: KL divergence of each model relative to the Baseline (cross-seed mean $\pm$ cross-seed std).

In terms of update magnitude, all fine-tuned models have KL significantly greater than zero relative to the Baseline, the minimum being 2.8 nats, which is about 0.05 nats

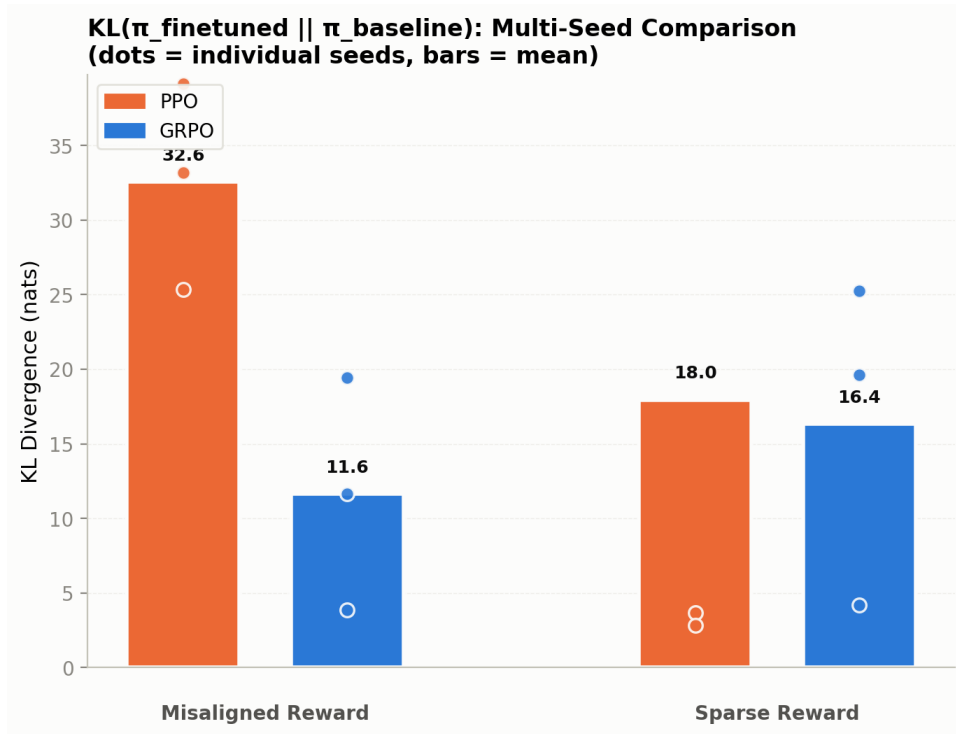


Figure 4.2: Action-distribution KL divergence of each model relative to the Baseline, over 3 seeds; the bar height is the cross-seed mean and the dots are individual seeds.

per dimension averaged over the 61-dimensional action space, showing that both types of policies undergo substantial updates and that GRPO’s stability does not stem from policy stagnation. Under the Misaligned condition, PPO’s update magnitude of 32.6 nats (0.53 per dimension) is about 3 times GRPO’s 11.6 nats (0.19 per dimension), and all three seeds are consistently high at 25.4–39.2, corresponding to its reward-hacking behaviour; GRPO’s update magnitude varies with seed in 3.8–19.4, but this variation does not translate into return fluctuation, reflecting KL-return decoupling. Under the Sparse condition, PPO’s update magnitude diverges sharply across seeds: seed 42’s KL explodes to 47.4 with a return collapse, while seeds 98/5075’s KL collapses to 2.8–3.7 with policy stalling; GRPO’s KL stays at 4.2–25.2 without collapse. Therefore, consistently across seeds and conditions, we observe that GRPO’s update magnitude is variable but its return is stable, while PPO’s update magnitude and return are tightly coupled and

its failure mode switches with the condition.

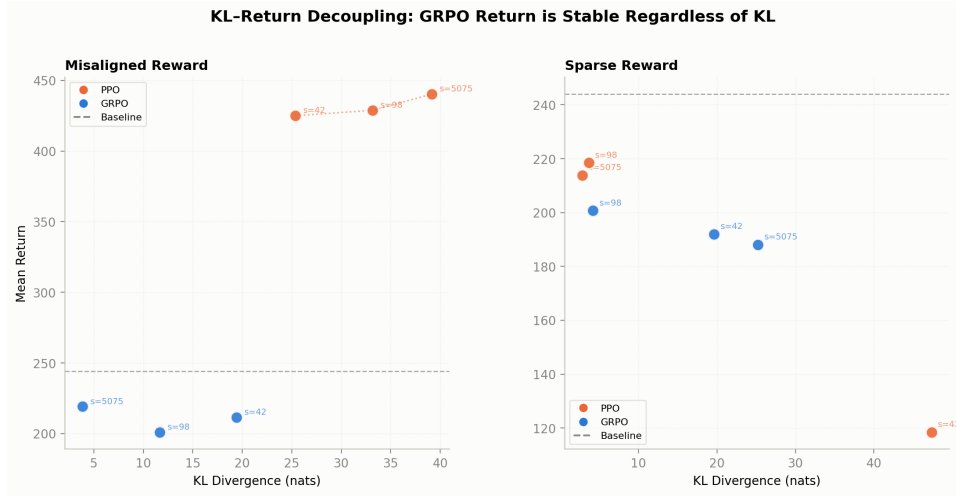


Figure 4.3: Scatter plot of KL divergence versus return, two panels, left = Misaligned Reward condition, right = Sparse Reward condition; the dashed line is the baseline reference.

Figure 4.3 shows GRPO’s KL-return decoupling: under both conditions, GRPO’s points cluster in a horizontal band around $return \approx 200$, with KL ranging from 3.8 to 25.2 while the return stays fixed; PPO behaves oppositely, with KL and return rising in tandem under the Misaligned condition and polarizing under the Sparse condition, where high KL corresponds to collapse and low KL to stalling. This contrast shows that GRPO’s policy-update magnitude can vary with seed and condition, but the update direction always stays aligned with the task goal without reward hacking; PPO’s update magnitude and direction are both driven by the reward signal and can easily become uncontrolled under unreliable rewards. Quantitatively, under the Sparse condition PPO’s cross-seed KL bandwidth of 44.6 nats corresponds to a return bandwidth of 100.1, while GRPO’s KL bandwidth of 21.0 nats corresponds to a return bandwidth of only 12.6: the seed-to-seed difference in update magnitude hardly propagates to GRPO’s performance difference, yet significantly propagates to PPO’s performance difference.

The cross-seed std of PPO (Sparse)’s action offset reaches 1.30, far higher than the other

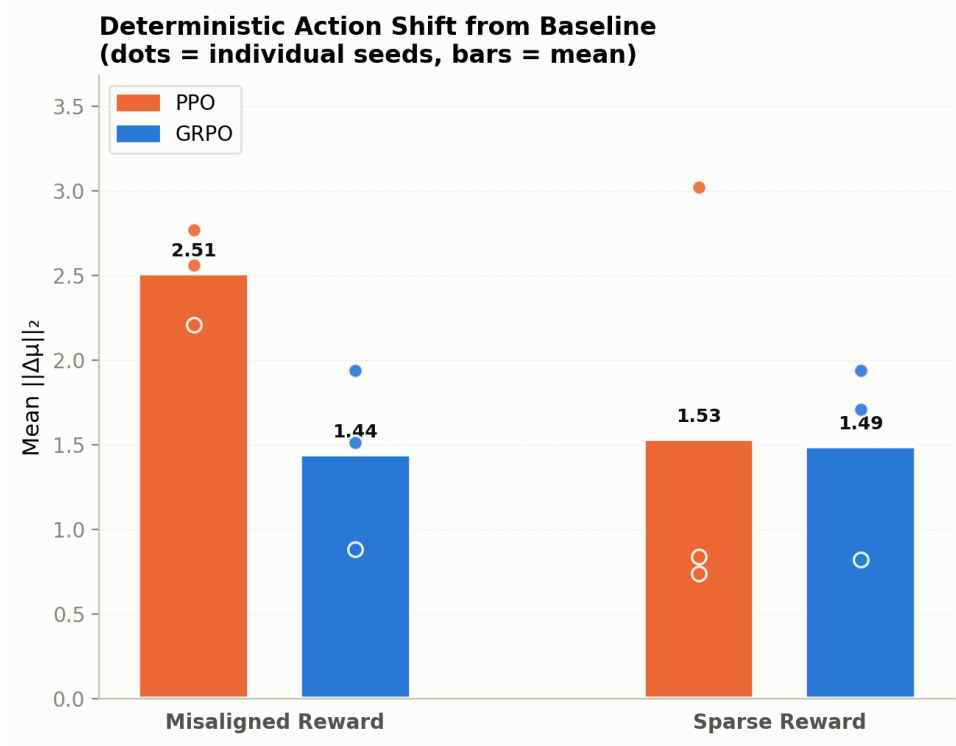


Figure 4.4: L2 distance of each model’s deterministic action mean relative to the Baseline, cross-seed mean, with dots for individual seeds.

configurations; GRPO’s action-offset means under the two conditions are 1.44 and 1.49 respectively, almost identical.

PPO (Misaligned) consistently reduces entropy across all seeds, with a shift of -0.05 to -0.08 ; this is one of the signatures of reward-hacking strategies: locking into a specific action pattern and reducing exploration to sustain a high return. PPO (Sparse)’s $\log\sigma$ change is highly inconsistent across seeds, with a cross-seed std of 0.028, consistent with its seed-to-seed return fluctuation. GRPO (Sparse)’s $\log\sigma$ is almost unchanged across the three seeds, with a shift of $+0.002 \pm 0.001$, maintaining the same exploration level as the Baseline; this may be a key factor enabling GRPO to avoid deterministic collapse under extremely sparse rewards.

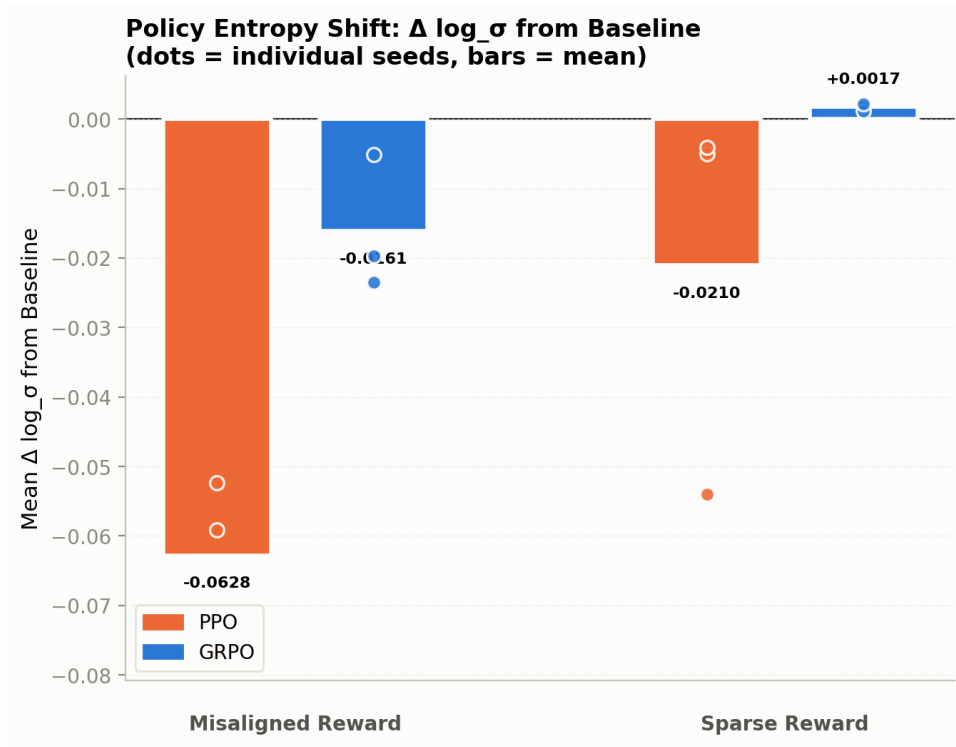


Figure 4.5: Average $\log\text{-}\sigma$ shift of each model relative to the Baseline, cross-seed mean, with dots for individual seeds.

4.2.3 Parameter-Space Distance

The cross-seed std of PPO (Sparse)’s parameter distance reaches 1.36; seed 42’s parameter offset of 3.37 is about 3 times the 1.09/0.96 of seeds 98/5075. GRPO’s parameter-distance means under the two conditions are 1.83 and 1.43. This cross-seed variation pattern is consistent with the action-distribution KL; for example, GRPO (Misaligned)’s KL std of 7.8 corresponds to a parameter-distance std of 0.63, the two metrics reflecting the same update characteristics from different levels.

4.2.4 VecNormalize Drift

The VecNormalize observation statistics drift as the policy distribution changes during training; it must be confirmed that this does not constitute an alternative explanation

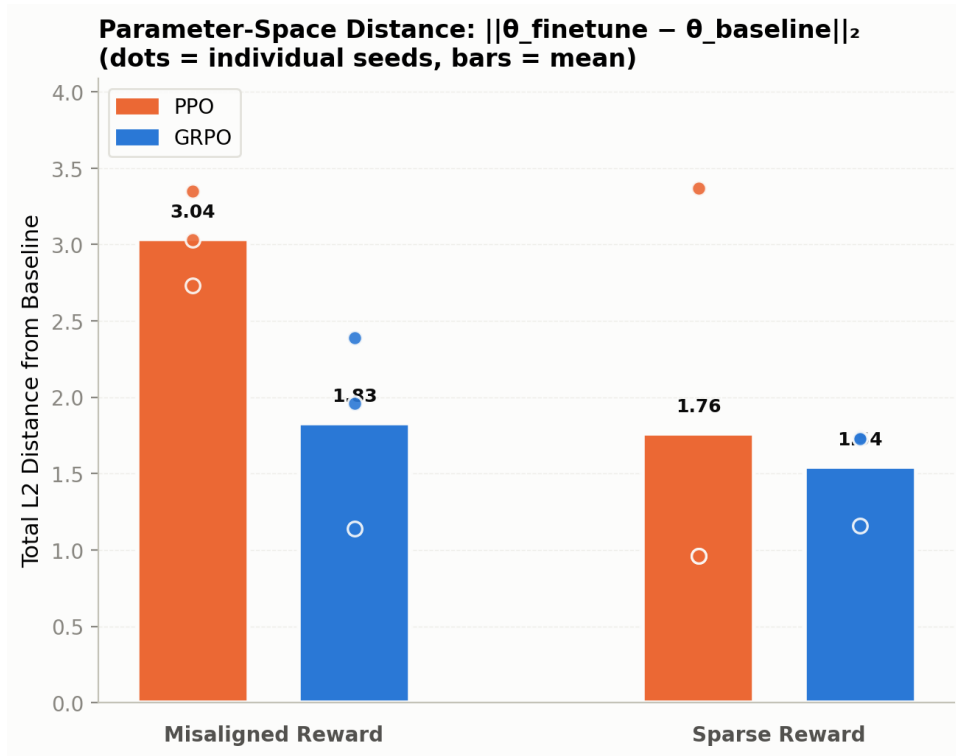


Figure 4.6: Parameter-space L2 distance of each model relative to the Baseline, cross-seed mean, with dots for individual seeds.

for the above conclusions. The drift magnitude across the four configurations is broadly similar: the cross-seed mean of $\|\Delta_{\text{mean}}\|$ ranges over 0.19–0.35 and $\|\Delta_{\text{var}}\|$ over 13–24, with only PPO (Sparse) having a higher cross-seed dispersion, with a cross-seed std of 22.1 for $\|\Delta_{\text{var}}\|$. The drift does not exhibit a systematic pattern matching the KL or return differences, and does not change the conclusions of Sections 4.2.2–4.2.3.

4.2.5 Summary

The evidence across seeds points consistently to the following: PPO either reward-hacks, or collapses or stalls, when rewards are unreliable, and this behaviour is reproducible across seeds; GRPO, by contrast, has substantial policy updates while its return stays stable, i.e., KL-return decoupling. Multiple metrics at the level of the action distribution,

Model	$\ \Delta\text{mean}\ $	$\ \Delta\text{var}\ $
PPO (Misaligned)	0.24 ± 0.04	16.3 ± 3.5
GRPO (Misaligned)	0.19 ± 0.11	13.4 ± 8.3
PPO (Sparse)	0.24 ± 0.29	17.5 ± 22.1
GRPO (Sparse)	0.35 ± 0.18	24.3 ± 12.8

Table 4.6: VecNormalize observation-statistic drift (cross-seed mean $\pm$ cross-seed std).

the parameter space and the exploration level corroborate each other, and this contrast does not depend on the checkpoint-selection metric.

4.3 Ablation Study: The Role of the KL Penalty

The standard GRPO adds a KL penalty relative to a reference policy in its loss, whereas the GRPO in our main experiments follows Doorman’s implementation and removes this penalty. To confirm that the stability is attributable to group-relative advantage rather than to the absence of the KL penalty, we add a GRPO ablation with the KL penalty, with coefficient $\beta = 0.01$ and all other settings identical to the main experiments, covering the 3 training seeds and the two reward conditions. The results are:

Metric	Baseline	GRPO	GRPO+KL	PPO
Return (Misaligned)	244.0	210.5 ± 9.1	222.4 ± 16.4	431.4 ± 8.0
Return (Sparse)	244.0	193.6 ± 6.5	217.9 ± 15.5	183.6 ± 56.5
door share (Misaligned)	49.8%	43.7%	49.2%	58.5%
door share (Sparse)	49.8%	41.1%	48.3%	35.9%
Success (Misaligned)	5.0%	$1.3\% \pm 1.2\%$	3.0%	$0.0\% \pm 0.0\%$
Success (Sparse)	5.0%	$1.3\% \pm 1.5\%$	1.7%	$0.3\% \pm 0.6\%$

Table 4.7: KL-penalty ablation results. The GRPO+KL column reports single-seed mean values (returns, success rates, door shares) across 3 seeds.

The GRPO with the KL penalty does not reward-hack under either condition: the door-openness shares of 49.2% (Misaligned) and 48.3% (Sparse) are basically level with the baseline’s 49.8%, whereas PPO’s corresponding shares are 58.5% and 35.9%; the returns of 222 and 218 are close to the baseline and no lower than the no-KL version (210 and

194); the success rates of 3.0% and 1.7% are no worse than the no-KL version. This shows that regardless of whether the KL penalty is present, GRPO does not amplify a single reward component, and that the stability stems from the group-relative advantage mechanism itself.

The actual role of the KL penalty is to tighten the update magnitude rather than to decide whether hacking occurs. The KL of GRPO+KL relative to the baseline is 3.5 nats and its parameter distance is 1.33, both clearly smaller than the no-KL version’s 11.6 nats and 1.83, but still significantly greater than zero (0.06 nats per dimension), showing that its policy still has substantial updates rather than stagnation; its return under the Sparse condition is actually higher (218 vs. 194) and it does not fail to learn because of the KL constraint.

4.4 Cross-Task Validation: The Reach Task

To further verify the generality of the above conclusions, we run additional experiments on the Reach task of HumanoidBench (‘h1hand-reach-v0’). The Reach task requires the H1 robot’s left hand to reach a target position, using a sparse binary success reward exactly like the Door Sparse condition: intermediate steps give a reward of 0, and a reward of 1.0 is given only at episode termination if the hand–target distance is less than 0.05 m. Both PPO and GRPO are initialized from the same pretrained Reach model with the same hyperparameters, and trained for 2M steps on a single seed.

The Reach task’s sparse reward is more extreme than Door’s: Door’s reward function still has a weighted structure of 6 components, whose values, although not aligned with success, provide implicit behavioural gradients; Reach’s reward contains only a single terminal binary signal, with intermediate steps entirely zero. The results are:

Because the evaluation dimensions of the pretrained Reach model and the fine-tuned models are incompatible, we cannot directly quantify how much of the pretrained suc-

Metric	PPO (Sparse)	GRPO (Sparse)
Success rate	0.0%	1.0%
Mean return	418.1 ± 320.9	712.6 ± 232.3

Table 4.8: Results on the Reach task.

cess rate the fine-tuning recovers; we therefore compare only the relative performance of the two fine-tuning strategies: GRPO’s return mean is higher, about 70% above PPO, with a smaller variance (from ± 321 down to ± 232), and it is the only model with at least one success across the 100 evaluation episodes. Under the extreme condition of purely sparse, no-intermediate-guidance rewards, GRPO’s relative advantage remains reproducible, consistent with the conclusions on the Door task. This result also reveals the boundary of GRPO’s robustness: when the reward signal completely lacks intermediate-state discrimination, the within-group normalization of group-relative advantage may degenerate into pure noise and GRPO also cannot learn effectively; nevertheless, its performance still surpasses PPO.

5.1 A Possible Mechanistic Explanation of GRPO’s Robustness

The core mechanism of GRPO is group-relative advantage: for K rollouts from the same initial state, the advantage function computes an individual’s relative performance against the within-group mean rather than the absolute reward value. The observations of Chapter 4 show that GRPO remains stable under both reward conditions, whereas PPO reward-hacks under the Misaligned condition and collapses or stalls under the Sparse condition. We explain this difference at the mechanistic level below.

Under the Misaligned Reward condition, PPO’s GAE mechanism may form a positive-feedback loop: the policy accidentally discovers that pushing and pulling the door increases the door-openness reward; GAE encodes this absolute increase as a positive gradient signal; the updated policy becomes better at pushing the door, the reward grows further, and the gradient becomes stronger, driving the behaviour to be even more extreme. GRPO’s group-relative mechanism may instead break this loop: when most samples in a group obtain a high door-openness reward, the within-group mean rises together and the individual’s relative advantage approaches zero; even a high absolute reward produces no strong gradient. Only behaviour that genuinely outperforms the other samples in the group is reinforced. Because the passing-the-door reward is extremely hard to obtain, no sample can stand out on that dimension, so the policy does not over-favour any single component; this is consistent with the observation in Section 4.2.1 that GRPO does not amplify the door-openness component.

Under the Sparse Reward condition, PPO faces a double dilemma: the reward-hacking behaviour learned under the Misaligned condition no longer yields any return, and the

sparse binary signal provides too little information to guide the policy out of its erroneous behavioural pattern. Under sparse rewards its GAE advantage estimate is either too noisy (causing deterministic collapse) or too weak (causing gradient vanishing), and the common root of the two failure modes observed in Section 4.2.2 is the lack of a stable signal scale. In contrast, GRPO’s within-group normalization always scales the advantage signal to a stable magnitude independent of the absolute scale of the external reward; since the policy is not locked into hacking behaviour under the Misaligned condition, it does not need to recover and can adapt to the Sparse condition directly, as shown by the cross-condition comparison in Section 4.1.

In addition, GRPO’s $\log\sigma$ remains conservative under the Sparse condition while PPO’s exploration level diverges strongly across seeds (Section 4.2.2). This suggests that the conservativeness of $\log\sigma$ may be a key factor enabling GRPO to avoid both deterministic collapse and gradient vanishing under extremely sparse rewards, although a causal verification of this hypothesis is left to future work.

5.2 Limitations

Weak baseline. The baseline’s 5% success rate is low and may limit the upper bound of fine-tuning performance. Future work could use a stronger baseline (e.g., a model pretrained with imitation learning or more rounds of curriculum training) to verify the generality of our conclusions under a stronger baseline.

State-distribution bias in KL divergence. Our KL-divergence analysis is based on 1000 observations collected under the baseline policy’s rollout distribution. If a fine-tuned policy drives the robot into states the baseline never visited (e.g., PPO’s extreme door-manipulation behaviour), the KL divergence may be systematically underestimated, because the policy differences on those out-of-distribution states are not sampled. In practice this limitation means the true KL difference between PPO and GRPO may be

larger than reported here. Future work should use a mixture of rollouts from multiple policies to cover a more complete state distribution.

Hyperparameter sensitivity of GRPO. Limited by single-GPU memory, we use only one set of GRPO hyperparameters: group size $K = 4$ and learning rate 3×10^{-5} . The sensitivity of GRPO to the group size remains to be studied; a larger K may enhance the statistical stability of within-group normalization, but may also weaken the discriminative power of the relative advantage.

Limited verification of the mechanistic explanation. The KL ablation in Section 4.3 rules out the influence of the “absence of the KL penalty” design choice on stability, but PPO and GRPO also differ in the presence of a critic, and the causal role of group-relative advantage has not been fully separated by a dedicated control experiment. Future work could further verify this via a critic-free PPO or a critic-based GRPO control.

6 | Conclusion

In scenarios with sparse rewards or unreliable reward functions, the choice of the RL algorithm directly affects the safety and reliability of the policy. Through systematic experiments on the HumanoidBench Door task, this thesis reveals fundamental differences between PPO and GRPO when facing such reward conditions:

1. **PPO engages in reward hacking when rewards are unreliable.** When the dense reward is not fully aligned with the true objective, PPO’s return soars from a baseline of 244 to 431 (+77%), while the success rate drops from 5.0% to 0.0%; its GAE advantage estimate amplifies the reward components weakly related to the success condition and suppresses the component related to the true goal. After removing the unreliable dense reward, PPO’s policy completely collapses, with the return falling below 52% of the baseline level.
2. **GRPO remains stable under both conditions, and its stability stems from KL-return decoupling rather than policy stagnation.** GRPO’s return stays within a narrow band of 188–219 across all seeds; the update magnitude (KL 3.8–25.2 nats, parameter distance 0.8–2.4) varies with seed and condition, but the return does not drift with the update magnitude. Under extremely sparse rewards, GRPO maintains the same exploration level as the Baseline ($\log \sigma$ shift of only +0.002) and avoids deterministic collapse. In contrast, PPO’s KL and return are tightly coupled: under the Misaligned condition, higher KL means more severe reward hacking; under the Sparse condition, it exhibits two failure modes, namely KL-explosion deterministic collapse and KL-collapse gradient vanishing.
3. **Reward decomposition and KL-penalty ablation jointly diagnose the source of stability.** Component-wise reward decomposition shows that PPO’s return gain is concentrated in components weakly related to success such as door

openness, while the passing-the-door component that truly reflects task completion decreases; the KL-penalty ablation rules out the alternative explanation that “stability stems from the absence of the KL penalty” (Section 4.3). Together these lines of evidence point to the normalization effect of group-relative advantage: components that are “consistent across the group” do not generate an advantage signal, so GRPO does not systematically amplify any single reward component.

4. **Cross-task evidence and practical implications.** Additional experiments on the Reach task show that the conclusions are not limited to the Door task. In scenarios where reward design is imperfect or rewards are extremely sparse, we recommend GRPO as the fine-tuning starting point: its robustness to the reward structure does not depend on the setting of the KL penalty, and its performance lower bound is significantly better than PPO’s (Section 4.1). Model selection should jointly monitor reward component decomposition and success rate rather than return increases alone; in our experiments the 77% return growth of PPO was precisely a misleading signal for selection, while selecting checkpoints by success rate yields conclusions consistent with the main experiments (Section 4.1). The applicability boundary of this recommendation is: when the reward completely lacks intermediate discrimination, such as the purely binary reward of the Reach task, the within-group normalization of group-relative advantage degenerates into noise and GRPO’s advantage narrows (Section 4.4).

References

- [1] S. Ibrahim, M. Mostafa, A. Jnadi, H. Salloum, and P. Osinenko, “Comprehensive overview of reward engineering and shaping in advancing reinforcement learning applications,” *IEEE Access*, vol. 12, pp. 175473–175500, 2024.
- [2] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Pieter Abbeel, and W. Zaremba, “Hindsight experience replay,” *Advances in neural information processing systems*, vol. 30, 2017.
- [3] G. Wang, M. Xin, W. Wu, Z. Liu, and H. Wang, “Learning of long-horizon sparse-reward robotic manipulator tasks with base controllers,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 3, pp. 4072–4081, 2022.
- [4] J. Yang, B. Zhu, J. Chen, and Y.-G. Jiang, “Actor-critic for continuous action chunks: A reinforcement learning framework for long-horizon robotic manipulation with sparse reward,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, pp. 18692–18700, 2026.
- [5] J. Karwowski, O. Hayman, X. Bai, K. Kiendlhofer, C. Griffin, and J. Skalse, “Goodhart’s law in reinforcement learning,” in *International Conference on Learning Representations*, vol. 2024, pp. 24546–24576, 2024.
- [6] R. Firoozi, J. Tucker, S. Tian, A. Majumdar, J. Sun, W. Liu, Y. Zhu, S. Song, A. Kapoor, K. Hausman, *et al.*, “Foundation models in robotics: Applications, challenges, and the future,” *The International Journal of Robotics Research*, vol. 44, no. 5, pp. 701–739, 2025.
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.

- [8] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.
- [9] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, *et al.*, “Deepseekmath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [10] J. Yao, Y. Wang, A. Zhang, Z. Sun, S. Wang, L. Mei, Y. Ge, and S. Liu, “Multimodal reward hacking in reinforcement learning,” *arXiv preprint arXiv:2607.09492*, 2026.
- [11] C. Sferrazza, D.-M. Huang, X. Lin, Y. Lee, and P. Abbeel, “Humanoidbench: Simulated humanoid benchmark for whole-body locomotion and manipulation,” *arXiv preprint arXiv:2403.10506*, 2024.
- [12] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in ai safety,” *arXiv preprint arXiv:1606.06565*, 2016.
- [13] J. Skalse, N. Howe, D. Krasheninnikov, and D. Krueger, “Defining and characterizing reward gaming,” *Advances in neural information processing systems*, vol. 35, pp. 9460–9471, 2022.
- [14] T. Everitt, V. Krakovna, L. Orseau, M. Hutter, and S. Legg, “Reinforcement learning with a corrupted reward channel,” *arXiv preprint arXiv:1705.08417*, 2017.
- [15] A. Pan, K. Bhatia, and J. Steinhardt, “The effects of reward misspecification: Mapping and mitigating misaligned models,” *arXiv preprint arXiv:2201.03544*, 2022.
- [16] X. Wang, M. Tian, Y. Zeng, Z. Huang, J. Yuan, B. Chen, J. Xu, M. Zhou, W. Liu, M. Wu, *et al.*, “Reward hacking in the era of large models: Mechanisms, emergent misalignment, challenges,” *arXiv preprint arXiv:2604.13602*, 2026.

- [17] M. Vecerik, T. Hester, J. Scholz, F. Wang, O. Pietquin, B. Piot, N. Heess, T. Rothörl, T. Lampe, and M. Riedmiller, “Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards,” *arXiv preprint arXiv:1707.08817*, 2017.
- [18] Z. Xie, P. Clary, J. Dao, P. Morais, J. Hurst, and M. Panne, “Learning locomotion skills for cassie: Iterative design and sim-to-real,” in *Conference on Robot Learning*, pp. 317–329, PMLR, 2020.
- [19] J. Hare, “Dealing with sparse rewards in reinforcement learning,” *arXiv preprint arXiv:1910.09281*, 2019.
- [20] C. Schwarke, V. Klemm, M. Van der Boon, M. Bjelonic, and M. Hutter, “Curiosity-driven learning of joint locomotion and manipulation tasks,” in *7th Annual Conference on Robot Learning*, 2023.
- [21] N. Hansen, J. SV, V. Sobal, Y. LeCun, X. Wang, and H. Su, “Hierarchical world models as visual whole-body humanoid controllers,” in *International Conference on Learning Representations*, vol. 2025, pp. 62175–62195, 2025.
- [22] H. Li, Y. Zuo, J. Yu, Y. Zhang, Y. Zhaohui, K. Zhang, X. Zhu, Y. Zhang, T. Chen, G. Cui, *et al.*, “Simplevla-rl: Scaling vla training via reinforcement learning,” in *International Conference on Learning Representations*, vol. 2026, pp. 125846–125869, 2026.
- [23] T. Zhang, C. Yu, S. Su, and Y. Wang, “Reinflow: Fine-tuning flow matching policy with online reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 106282–106319, 2026.
- [24] W. Huang, Z. Li, H. Liu, B. Hou, Y. Su, and J. Zhang, “Towards bridging the gap between large-scale pretraining and efficient finetuning for humanoid control,”

- in *International Conference on Learning Representations*, vol. 2026, pp. 134640–134668, 2026.
- [25] I. Radosavovic, S. Kamat, T. Darrell, and J. Malik, “Learning humanoid locomotion over challenging terrain,” *arXiv preprint arXiv:2410.03654*, 2024.
- [26] K. Hu, H. Shi, Y. He, W. Wang, C. K. Liu, and S. Song, “Robot trains robot: Automatic real-world policy adaptation and learning for humanoids,” *arXiv preprint arXiv:2508.12252*, 2025.
- [27] Z. Gu, J. Li, W. Shen, W. Yu, Z. Xie, S. McCrory, X. Cheng, A. Shamsah, R. Griffin, C. K. Liu, *et al.*, “Humanoid locomotion and manipulation: Current progress and challenges in control, planning, and learning,” *IEEE/ASME Transactions on Mechatronics*, vol. 31, no. 2, pp. 2300–2330, 2026.
- [28] O. E. Mansouri, F. A. Izzati, M. E. A. Seddik, and S. Lahlou, “Noise-corrected grpo: From noisy rewards to unbiased gradients,” *arXiv preprint arXiv:2510.18924*, 2025.
- [29] R. Khanda, M. Baqar, S. Chakrabarti, and S. Changdar, “Extending group relative policy optimization to continuous control: A theoretical framework for robotic reinforcement learning,” *arXiv preprint arXiv:2507.19555*, 2025.
- [30] B. L. de Oliveira, F. V. Frujeri, M. P. Queiroz, L. G. Martins, T. W. d. L. Soares, and L. C. Melo, “Learning without critics? revisiting grpo in classical reinforcement learning environments,” *arXiv preprint arXiv:2511.03527*, 2025.
- [31] H. Xue, T. He, Z. Wang, Q. Ben, W. Xiao, Z. Luo, X. Da, F. Castañeda, G. Shi, S. Sastry, *et al.*, “Opening the sim-to-real door for humanoid pixel-to-action policy transfer,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6642–6652, 2026.

A | Source Code

Source code for all of the methods implemented in Chap. 3 for the project can be found in the GitHub repository:

[link](#).

Project files can be found on Google Drive [link](#)

B | Project Introduction Video

A short video presentation, introducing background, aims and organisation of the project,
as of 30th June 2022:

[link](#)

